Assignment 2

October 19, 2026

1 EN3160 Assignment 02: Fitting and Alignment

1.1 230180F - Minaya Fernando

Github Repo link - https://github.com/Minu-Fernando/EN3160.git

[2]:
```
import numpy as np
import cv2 as cv
import matplotlib.pyplot as plt
from scipy.ndimage import gaussian_laplace, maximum_filter
from scipy.optimize import minimize
from pathlib import Path

image_dir = Path('a2images')
if not image_dir.exists():
    image_dir = Path('Assignment 2/a2images')
assert image_dir.exists(), 'Cannot find a2images'

def show(image, title, ax):
    if image.ndim == 2:
        ax.imshow(image, cmap='gray', vmin=0, vmax=255)
    else:
        ax.imshow(cv.cvtColor(image, cv.COLOR_BGR2RGB))
    ax.set_title(title)
    ax.axis('off')
```

1.2 Question 1 - Sunflower blob detection

I have the scale-normalized Laplacian of Gaussian, $\sigma^2\nabla^2(G_\sigma * I)$. Positive peaks detect the dark flower centres. A candidate must be a local maximum in position and scale. Its radius is estimated as $r = \sqrt{2}\sigma$. I have also checkd for yellow pixels around each candidate to reduce detections on leaves and the sky. No Hough circle detector is used.

[30]:
```
flowers = cv.imread(str(image_dir / 'the_berry_farms_sunflower_field.jpeg'), cv.
  ↪IMREAD_REDUCED_COLOR_4)
gray = cv.cvtColor(flowers, cv.COLOR_BGR2GRAY) / 255.0
hsv = cv.cvtColor(flowers, cv.COLOR_BGR2HSV)
sigmas = np.arange(2, 16.1, 0.5)   #range of sigma values to use for LoG
response = np.stack([s**2 * gaussian_laplace(gray, s) for s in sigmas])
```

```
peaks = (response == maximum_filter(response, size=(3, 3, 3))) & (response > 0.
 ↪24) # The blob detection in both scale and spatial dimensions.
# The threshold is set to 0.055 to filter out weak responses.
# peaks is a boolean array indicating the locations of local maxima in the␣
 ↪response array, which correspond to potential blob centers. ({scale index},␣
 ↪y, x)

peaks[[0, -1]] = False  # removing peaks in the fist and last scale to avoid␣
 ↪edge effects because it doesnt hv neighboring sigma layes to compare with.
candidates = sorted(np.argwhere(peaks), key=lambda p: response[tuple(p)],␣
 ↪reverse=True)
yy, xx = np.indices(gray.shape)
circles = []
for k, y, x in candidates:
    radius = np.sqrt(2) * sigmas[k]
    if x < radius or y < radius or x+radius >= gray.shape[1] or y+radius >=␣
 ↪gray.shape[0]:
        continue
    distance = np.hypot(xx-x, yy-y)
    if any(np.hypot(x-c[0], y-c[1]) < 0.7*(radius+c[2]) for c in circles):
        continue
    circles.append((x, y, radius, sigmas[k]))

fig, axes = plt.subplots(1, 2, figsize=(13, 6))
show(flowers, 'Original at quarter size', axes[0])
show(flowers, 'LoG detections', axes[1])
for x, y, radius, sigma in circles:
    axes[1].add_patch(plt.Circle((x, y), radius, fill=False, color='red',␣
 ↪linewidth=1))
plt.tight_layout()
plt.show()
print('Sigma range: 2 to 16 pixels, step 0.5; response threshold: 0.24')
print('Number of detections:', len(circles))
print('Largest circles: x, y, radius, sigma')
for circle in sorted(circles, key=lambda c: c[2], reverse=True)[:5]:
    print(tuple(round(float(value), 2) for value in circle))
```




```
Sigma range: 2 to 16 pixels, step 0.5; response threshold: 0.24
Number of detections: 34
Largest circles: x, y, radius, sigma
(338.0, 165.0, 18.38, 13.0)
(293.0, 169.0, 16.97, 12.0)
(282.0, 338.0, 9.9, 7.0)
(223.0, 174.0, 9.19, 6.5)
(106.0, 255.0, 8.49, 6.0)
```

1.3 Question 2 - RANSAC line and circle fitting

1.3.1 Generate the noisy points

This follows the data generation in the assignment. The seed makes the results repeatable. The fitting steps only use `X`; the original labels are used later for evaluation.

[32]:
```python
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)

N = 100
half_n = N // 2

# Noisy circle points
r = 10
x0_gt, y0_gt = 2, 3
s = r / 16

t = np.random.uniform(0, 2 * np.pi, half_n)
```

```
n = s * np.random.randn(half_n)

x = x0_gt + (r + n) * np.cos(t)
y = y0_gt + (r + n) * np.sin(t)
X_circ = np.hstack((x.reshape(half_n, 1), y.reshape(half_n, 1)))

# Noisy line points
s = 1.0
m, b = -1, 2

x = np.linspace(-12, 12, half_n)
y = m * x + b + s * np.random.randn(half_n)
X_line = np.hstack((x.reshape(half_n, 1), y.reshape(half_n, 1)))

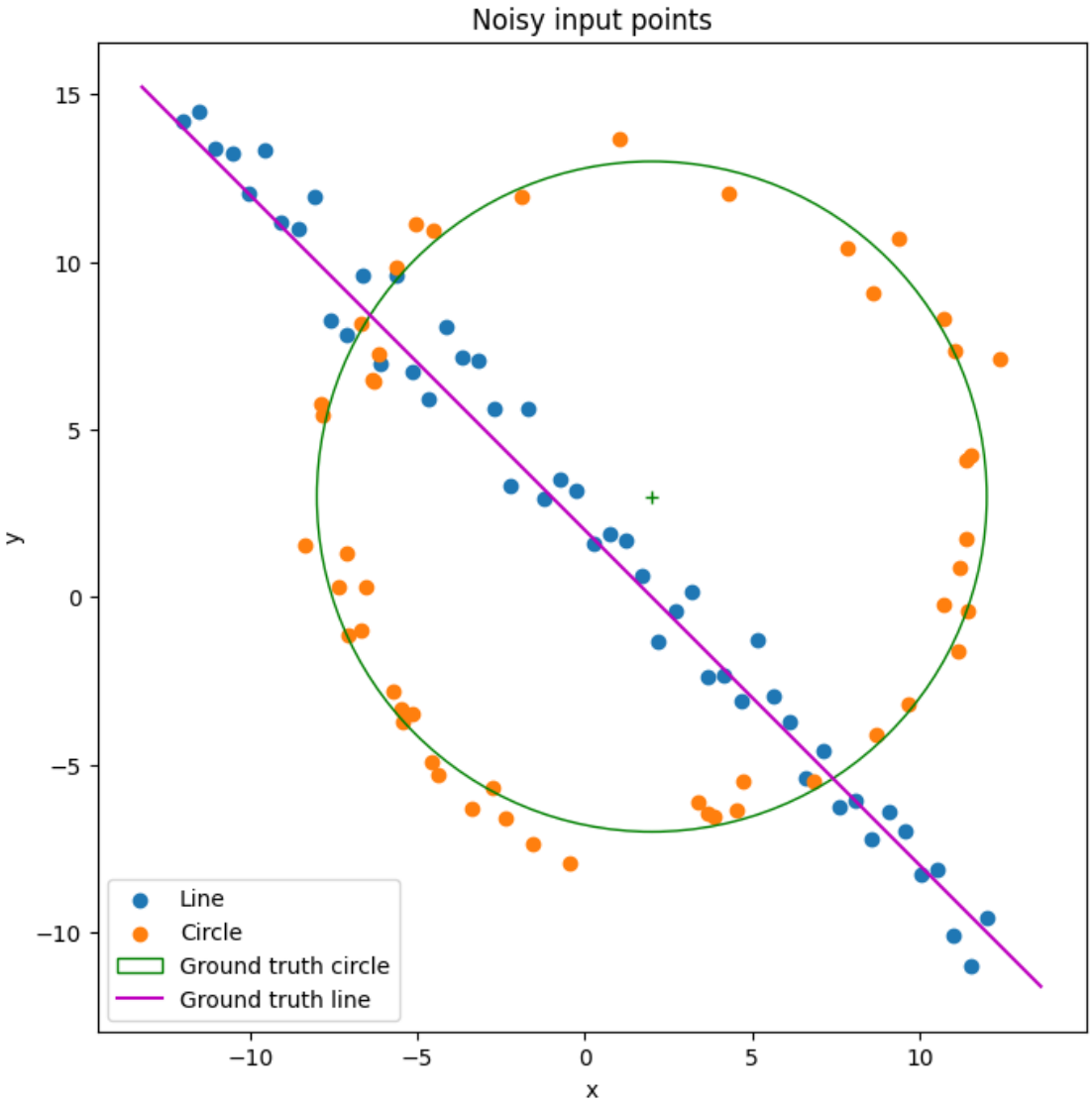
# All points
X = np.vstack((X_circ, X_line))

# Plot points
fig, ax = plt.subplots(figsize=(8, 8))
ax.scatter(X_line[:, 0], X_line[:, 1], label="Line")
ax.scatter(X_circ[:, 0], X_circ[:, 1], label="Circle")

# Ground truth circle and centre
circle_gt = plt.Circle(
    (x0_gt, y0_gt),
    r,
    color="g",
    fill=False,
    label="Ground truth circle"
)
ax.add_patch(circle_gt)
ax.plot(x0_gt, y0_gt, "+", color="g")

# Ground truth line
x_min, x_max = ax.get_xlim()
x_ = np.array([x_min, x_max])
y_ = m * x_ + b
ax.plot(x_, y_, color="m", label="Ground truth line")

ax.set(xlabel="x", ylabel="y", title="Noisy input points")
ax.set_aspect("equal")
ax.legend()
plt.show()
```



1.3.2 2(a) Fit the line first

The line is $ax + by = d$ with $a^2 + b^2 = 1$. Therefore $|ax + by - d|$ is the perpendicular distance. I sample two points at a time, use a threshold of 1.5, and require at least 35 inliers. The perpendicular noise standard deviation is about $1/\sqrt{2}$, so 1.5 is roughly two standard deviations. The winning consensus is refitted using total least squares.

[41]:
```
line_threshold = 1.5
best_score = (-1, -np.inf)
for _ in range(2000):
    ids = rng.choice(len(X), 2, replace=False)
    delta = X[ids[1]] - X[ids[0]]
    normal = np.array([-delta[1], delta[0]])
```

```
    if np.linalg.norm(normal) < 1e-10:
        continue
    normal /= np.linalg.norm(normal)
    d = normal @ X[ids[0]]
    errors = np.abs(X @ normal - d)
    inliers = errors < line_threshold
    score = (int(inliers.sum()), -np.sum(errors[inliers]**2))
    if inliers.sum() >= 35 and score > best_score:
        best_score = score
        line_sample = ids.copy()
        trial_line = np.r_[normal, d]
        line_consensus = inliers.copy()
assert best_score[0] >= 35, 'No line consensus found'
points = X[line_consensus]
centre = points.mean(axis=0)
_, _, vt = np.linalg.svd(points-centre)
normal = vt[-1]
d = normal @ centre
if d < 0:
    normal, d = -normal, -d
fitted_line = np.r_[normal, d]
line_inliers = np.abs(X @ normal-d) < line_threshold
print('Fitted [a, b, d]:', fitted_line)
print('Length of normal:', np.linalg.norm(normal))
print('Line inliers:', line_inliers.sum())
```

```
Fitted [a, b, d]: [0.71814154 0.69589707 1.52220339]
Length of normal: 1.0
Line inliers: 54
```

1.3.3 2(b) Fit a circle to the remaining points

Three non-collinear points define a trial circle. The residual is the absolute difference between a point's distance from the centre and the radius. I use a threshold of 1.5, at least 25 inliers and 3000 trials. The radial noise standard deviation is 0.625. Finally, SciPy minimizes squared radial errors on the consensus.

[34]:
```
def circle_from_three(points):
    p, q, s = points
    A = 2*np.array([q-p, s-p])
    if abs(np.linalg.det(A)) < 1e-8:
        return None
    centre = np.linalg.solve(A, [q@q-p@p, s@s-p@p])
    return np.r_[centre, np.linalg.norm(p-centre)]

def circle_ransac(points, generator):
    best_score = (-1, -np.inf)
    for _ in range(3000):
```

```
        ids = generator.choice(len(points), 3, replace=False)
        model = circle_from_three(points[ids])
        if model is None:
            continue
        errors = np.abs(np.linalg.norm(points-model[:2], axis=1)-model[2])
        mask = errors < 1.5
        score = (int(mask.sum()), -np.sum(errors[mask]**2))
        if mask.sum() >= 25 and score > best_score:
            best_score = score
            best = (model.copy(), ids.copy(), mask.copy())
    if best_score[0] < 25:
        raise RuntimeError('No circle consensus found')
    trial, ids, mask = best
    selected = points[mask]
    def objective(c):
        return np.sum((np.linalg.norm(selected-c[:2], axis=1)-c[2])**2)
    result = minimize(objective, trial, method='L-BFGS-B',
                      bounds=[(None, None), (None, None), (1e-6, None)])
    if not result.success:
        raise RuntimeError(result.message)
    fitted = result.x
    mask = np.abs(np.linalg.norm(points-fitted[:2], axis=1)-fitted[2]) < 1.5
    return trial, ids, fitted, mask

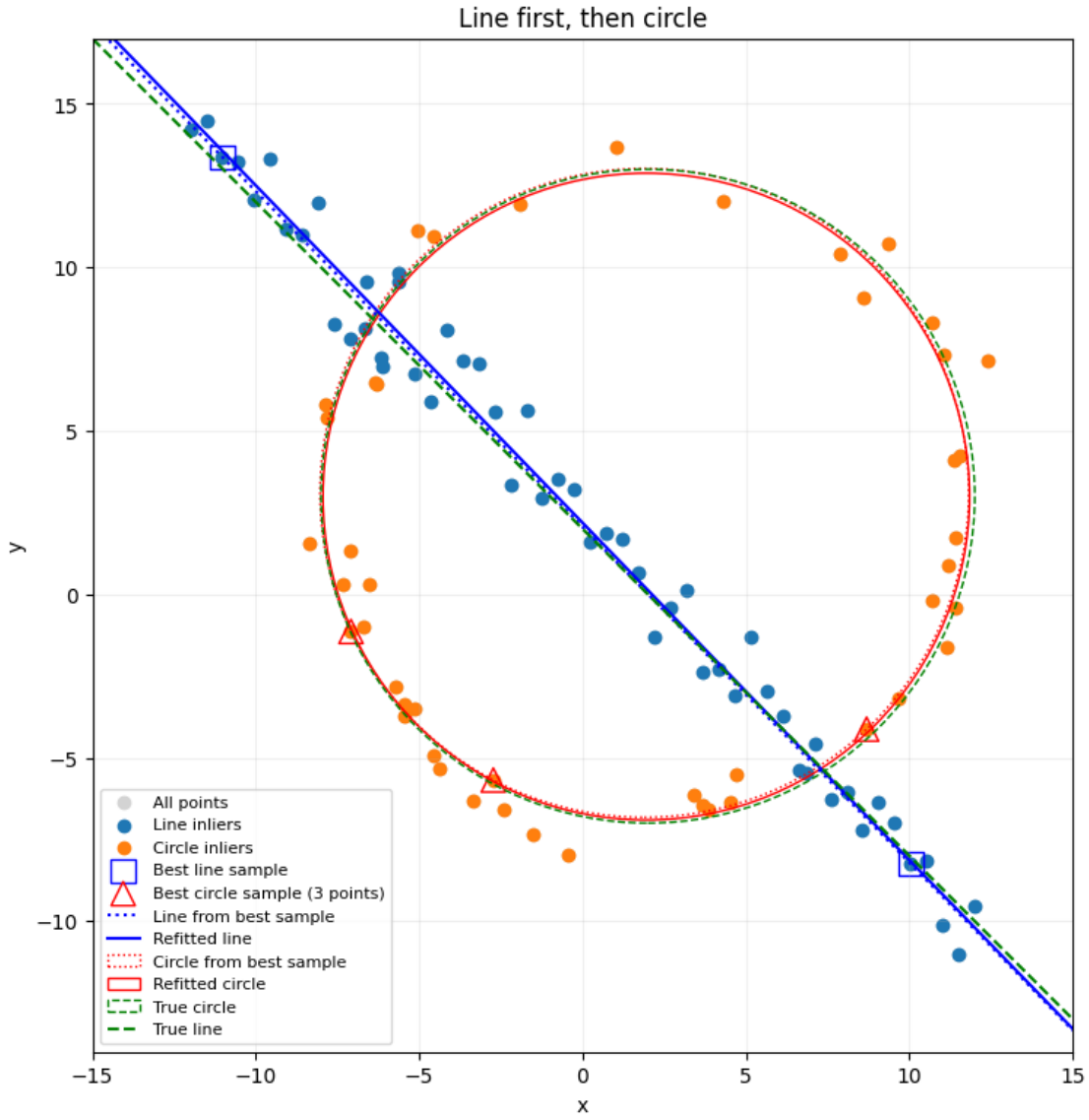
remaining_ids = np.flatnonzero(~line_inliers)
remaining = X[remaining_ids]
trial_circle, sample_ids, fitted_circle, circle_mask = circle_ransac(remaining,␣
 ↪rng)
circle_sample = remaining_ids[sample_ids]
circle_inliers = remaining_ids[circle_mask]
print('Circle [centre x, centre y, radius]:', fitted_circle)
print('Circle inliers:', len(circle_inliers), 'of', len(remaining))
print('RMS radial error:', np.sqrt(np.mean(
    (np.linalg.norm(X[circle_inliers]-fitted_circle[:2],␣
 ↪axis=1)-fitted_circle[2])**2)))
```

```
Circle [centre x, centre y, radius]: [1.94939851 2.98785106 9.89192183]
Circle inliers: 46 of 46
RMS radial error: 0.5924514945458131
```

1.3.4 2(c) Show samples, fitted models and inliers together

[35]:
```
fig, ax = plt.subplots(figsize=(10, 9))
ax.scatter(*X.T, color='lightgray', label='All points')
ax.scatter(*X[line_inliers].T, label='Line inliers')
ax.scatter(*X[circle_inliers].T, label='Circle inliers')
```

```
ax.scatter(*X[line_sample].T, marker='s', s=140, facecolors='none',␣
 ↪edgecolors='blue', label='Best line sample')
ax.scatter(*X[circle_sample].T, marker='^', s=140, facecolors='none',␣
 ↪edgecolors='red', label='Best circle sample (3 points)')
xx = np.linspace(-15, 15, 200)
for model, style, label in [(trial_line, 'b:', 'Line from best sample'),␣
 ↪(fitted_line, 'b-', 'Refitted line')]:
    a, b, d = model
    if abs(b) > 1e-8:
        ax.plot(xx, (d-a*xx)/b, style, label=label)
    else:
        ax.axvline(d/a, color='blue', label=label)
for model, colour, style, label in [(trial_circle, 'red', ':', 'Circle from␣
 ↪best sample'),
                                    (fitted_circle, 'red', '-', 'Refitted␣
 ↪circle'),
                                    ([2, 3, 10], 'green', '--', 'True circle')]:
    ax.add_patch(plt.Circle(model[:2], model[2], fill=False, color=colour,␣
 ↪linestyle=style, label=label))
ax.plot(xx, -xx+2, 'g--', label='True line')
ax.set(xlim=(-15, 15), ylim=(-14, 17), xlabel='x', ylabel='y', title='Line␣
 ↪first, then circle')
ax.set_aspect('equal')
ax.legend(fontsize=8)
ax.grid(alpha=0.2)
plt.show()
```



1.3.5 2(d) Fit the circle first

A very large circle can approximate a line over a short arc. This can cause circle RANSAC to pick line points. Even a correct circle can include line points near the intersections. The following cell checks what happens with this particular dataset.

[42]:
```
_, _, first_circle, first_mask = circle_ransac(X, np.random.default_rng(7))
print('Circle-first [x, y, radius]:', first_circle)
print('Consensus size:', first_mask.sum())
print('True circle points selected:', first_mask[:half_n].sum())
print('True line points selected:', first_mask[half_n:].sum())
```

```
Circle-first [x, y, radius]: [1.97898832 2.96633199 9.95821766]
Consensus size: 59
True circle points selected: 50
True line points selected: 9
```

Circle-first does not always fail. In this run it finds the true circle, but also includes some line points. Line-first also removes a few circle points near the intersections, so neither order perfectly separates the models.

1.4 Question 3 - Perspective warping and blending

[60]:
```
%matplotlib inline
```

[61]:
```
from pathlib import Path

folder = Path('a2images')
CLICK_FOUR_POINTS = True  # Set to True to select 4 points with your mouse
BLEND_ALPHA = 0.88        # 1.0 = opaque flag, 0.0 = original building

pairs = [
    ('Modern building + Japan', 'wadham.jpg', 'flag.png',
     [[370, 220], [620, 235], [614, 432], [361, 424]])
]

def read_rgb(path):
    im = cv.imread(str(path))
    if im is None:
        raise FileNotFoundError(f'Image not found: {path}')
    return cv.cvtColor(im, cv.COLOR_BGR2RGB)

def get_four_points(image, preset):
    if not CLICK_FOUR_POINTS:
        return np.float32(preset)

    # OpenCV handles mouse events locally, without VS Code's widget renderer.
    window = 'Select TL, TR, BR, BL | Enter: confirm | Right-click: undo | Esc:␣
 ↪cancel'
    height, width = image.shape[:2]
    scale = min(1.0, 1200 / width, 800 / height)
    display_size = (max(1, round(width * scale)), max(1, round(height * scale)))
    preview = cv.resize(cv.cvtColor(image, cv.COLOR_RGB2BGR), display_size)
    points = []

    def on_click(event, x, y, flags, param):
        if event == cv.EVENT_LBUTTONDOWN and len(points) < 4:
            if 0 <= x < display_size[0] and 0 <= y < display_size[1]:
                points.append((x, y))
        elif event == cv.EVENT_RBUTTONDOWN and points:
```

```
            points.pop()

    print('Select TL, TR, BR, BL in the image window, then press Enter. '
          'Right-click to undo; Esc to cancel.')
    cv.namedWindow(window, cv.WINDOW_AUTOSIZE)
    cv.setMouseCallback(window, on_click)
    try:
        while True:
            marked = preview.copy()
            for i, (x, y) in enumerate(points):
                cv.circle(marked, (x, y), 5, (0, 0, 255), -1)
                cv.putText(marked, str(i + 1), (x + 8, y - 8),
                           cv.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
            cv.imshow(window, marked)
            key = cv.waitKey(20) & 0xFF
            if key == 27 or cv.getWindowProperty(window, cv.WND_PROP_VISIBLE) <␣
↪1:
                raise RuntimeError('Point selection cancelled. Rerun this cell␣
↪to try again.')
            if key in (10, 13) and len(points) == 4:
                break
    finally:
        cv.destroyWindow(window)

    # Map preview coordinates back to the original image for the homography.
    dst = np.asarray(points, dtype=np.float32)
    dst *= np.float32([width / display_size[0], height / display_size[1]])
    print('Selected points:')
    print(dst)
    return dst
```

[62]:
```
def overlay_flag(building, flag, corners, alpha=0.88):
    h, w = flag.shape[:2]
    src = np.float32([[0, 0], [w-1, 0], [w-1, h-1], [0, h-1]])

    H = cv.getPerspectiveTransform(src, corners)
    out_size = (building.shape[1], building.shape[0])
    warped = cv.warpPerspective(flag, H, out_size)

    mask = np.full((h, w), 255, dtype=np.uint8)
    warped_mask = cv.warpPerspective(mask, H, out_size) / 255.0
    a = alpha * warped_mask[..., None]
    output = ((1-a)*building + a*warped).astype(np.uint8)
    return output, H
```

[63]:
```
results = []
```

```
for title, building_name, flag_name, preset in pairs:
    building = read_rgb(folder / building_name)
    flag = read_rgb(folder / flag_name)
    corners = get_four_points(building, preset)
    output, H = overlay_flag(building, flag, corners, 0.5)
    results.append((title, building, flag, corners, output, H))

fig, axes = plt.subplots(len(results), 3, figsize=(17, 4.5 * len(results)),␣
 ↪squeeze=False)
for i, (title, building, flag, corners, output, H) in enumerate(results):
    marked = building.copy()
    cv.polylines(marked, [corners.astype(np.int32)], True, (255, 230, 0), 3)
    for j, (x, y) in enumerate(corners.astype(int)):
        cv.circle(marked, (x, y), 7, (255, 60, 20), -1)
        cv.putText(marked, str(j+1), (x+9, y-9), cv.FONT_HERSHEY_SIMPLEX,
                   0.8, (255, 255, 255), 2)
    axes[i, 0].imshow(marked)
    axes[i, 0].set_title(f'{title}: selected 4 points')
    axes[i, 1].imshow(flag)
    axes[i, 1].set_title('Flag source image')
    axes[i, 2].imshow(output)
    axes[i, 2].set_title('Warped and blended result')
    for ax in axes[i]:
        ax.axis('off')
plt.tight_layout()
plt.show()
```

```
Select TL, TR, BR, BL in the image window, then press Enter. Right-click to
undo; Esc to cancel.
Selected points:
[[142. 204.]
 [521. 290.]
 [522. 463.]
 [132. 446.]]
```



A homography maps one plane onto another. Alpha blending lets wall texture show through the

overlay. It does not simulate shadows or illumination, so the result still looks edited. A single homography would not fit a curved surface.

1.5 Question 4 - PCB alignment

1.5.1 4(a) Detect SIFT features

AECC1 is the reference. Images are resized to a maximum side length of 1200 pixels to reduce processing time. All following pixel thresholds refer to this resolution.

[46]:
```
def read_pcb(path):
    image = cv.imread(str(path))
    if image is None:
        raise FileNotFoundError(path)
    scale = min(1, 1200/max(image.shape[:2]))
    return cv.resize(image, None, fx=scale, fy=scale, interpolation=cv.
 ↪INTER_AREA)

pcb_dir = image_dir / 'pcbs'
template = read_pcb(pcb_dir / 'AECC1.jpg')
sift = cv.SIFT_create(nfeatures=6000)
template_kp, template_des = sift.detectAndCompute(cv.cvtColor(template, cv.
 ↪COLOR_BGR2GRAY), None)
boards = []
for path in sorted(pcb_dir.glob('*.jpg')):
    if path.name == 'AECC1.jpg':
        continue
    image = read_pcb(path)
    kp, des = sift.detectAndCompute(cv.cvtColor(image, cv.COLOR_BGR2GRAY), None)
    boards.append(dict(name=path.name, image=image, kp=kp, des=des))
    print(path.name, ':', len(kp), 'keypoints')
print('AECC1 template:', len(template_kp), 'keypoints')
```

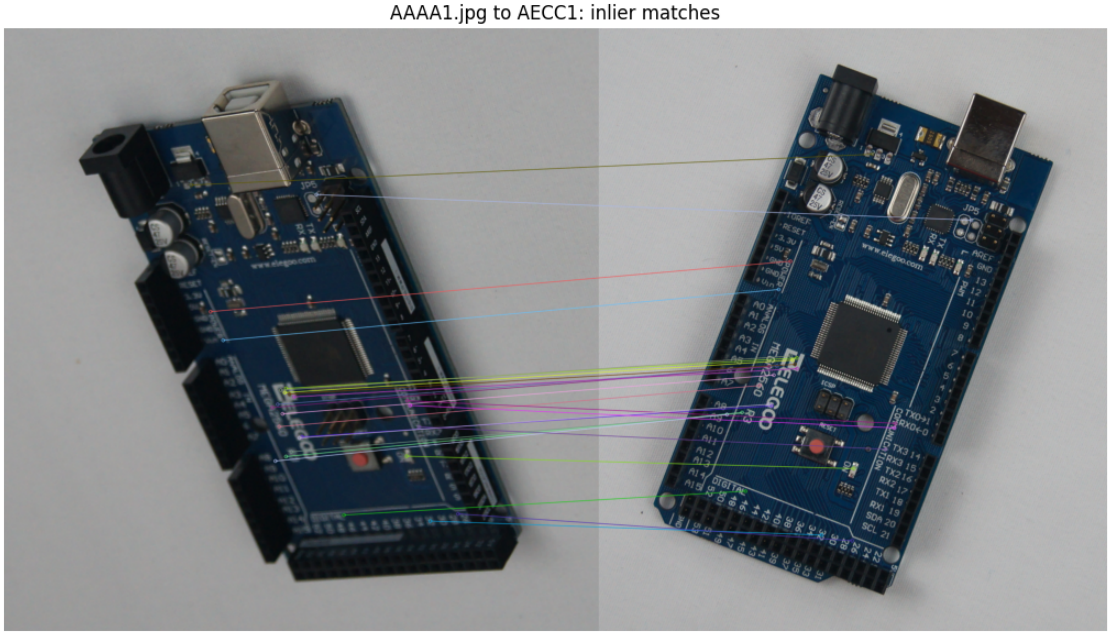
```
AAAA1.jpg : 1671 keypoints
ADAD1.jpg : 4384 keypoints
AECC2.jpg : 5461 keypoints
AECC3.jpg : 5302 keypoints
AECC4.jpg : 5639 keypoints
AECC1 template: 5492 keypoints
```

1.5.2 4(b) Match features and estimate homographies

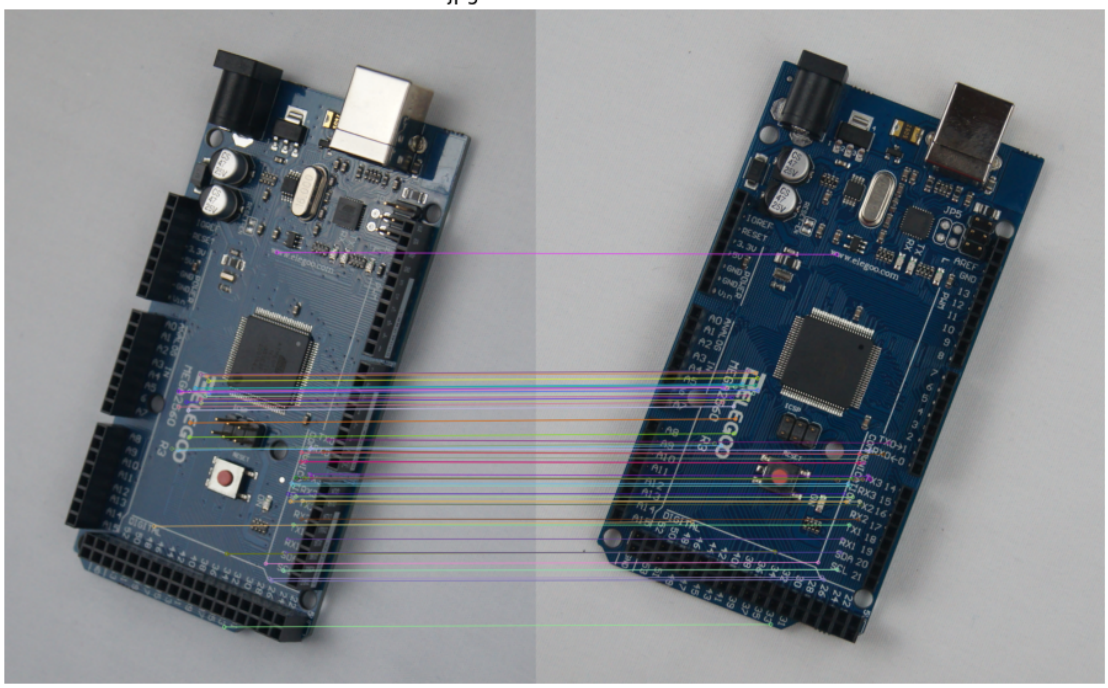
I use BFMatcher with Euclidean distance and a 0.75 ratio test to reject ambiguous matches. RANSAC estimates the image-to-template homography with a 4-pixel threshold. The figures show up to 60 of the best RANSAC inlier matches for every image.

[47]:
```
matcher = cv.BFMatcher(cv.NORM_L2)
for board in boards:
    if board['des'] is None:
```

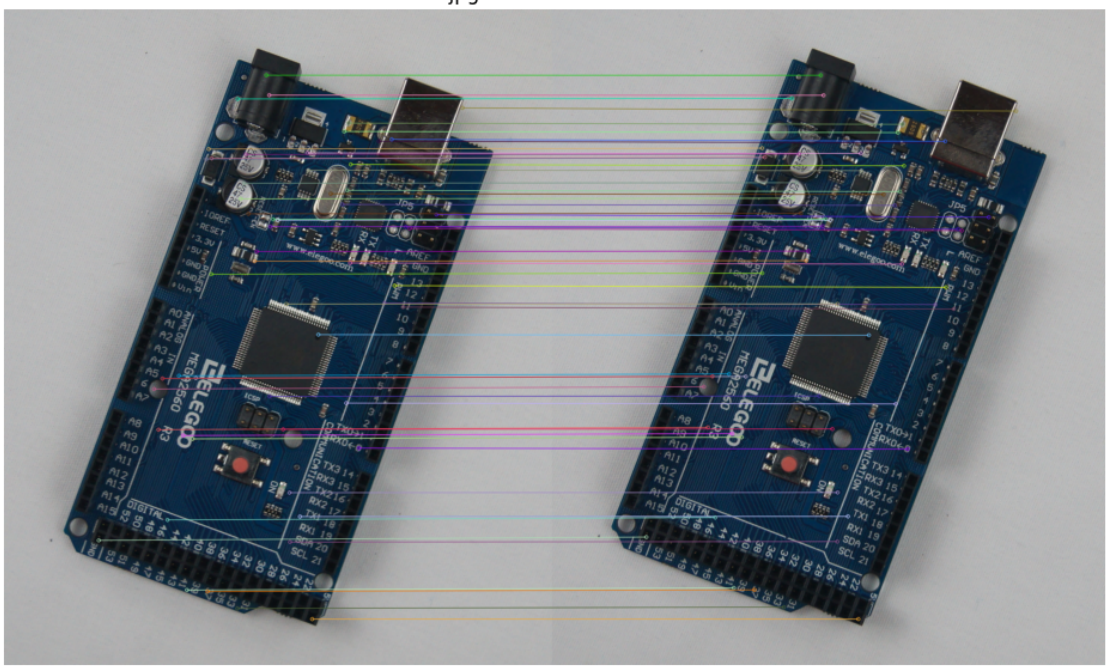
```
        raise RuntimeError('No descriptors: ' + board['name'])
    pairs = matcher.knnMatch(board['des'], template_des, k=2)
    good = [p[0] for p in pairs if len(p) == 2 and p[0].distance < 0.75*p[1].
↪distance]
    if len(good) < 4:
        raise RuntimeError('Not enough matches: ' + board['name'])
    src = np.float32([board['kp'][m.queryIdx].pt for m in good]).reshape(-1, 1,␣
↪2)
    dst = np.float32([template_kp[m.trainIdx].pt for m in good]).reshape(-1, 1,␣
↪2)
    cv.setRNGSeed(0)
    H, mask = cv.findHomography(src, dst, cv.RANSAC, 4.0)
    if H is None or mask.sum() < 4:
        raise RuntimeError('Homography failed: ' + board['name'])
    mask = mask.ravel().astype(bool)
    error = np.linalg.norm(cv.perspectiveTransform(src, H)-dst, axis=2).ravel()
    board.update(H=H, good=len(good), inliers=int(mask.sum()), error=np.
↪median(error[mask]))
    matches = sorted([m for m, keep in zip(good, mask) if keep], key=lambda m:␣
↪m.distance)[:60]
    drawing = cv.drawMatches(board['image'], board['kp'], template,␣
↪template_kp, matches, None,
                             flags=cv.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
    fig, ax = plt.subplots(figsize=(13, 6))
    show(drawing, board['name'] + ' to AECC1: inlier matches', ax)
    plt.tight_layout()
    plt.show()
```



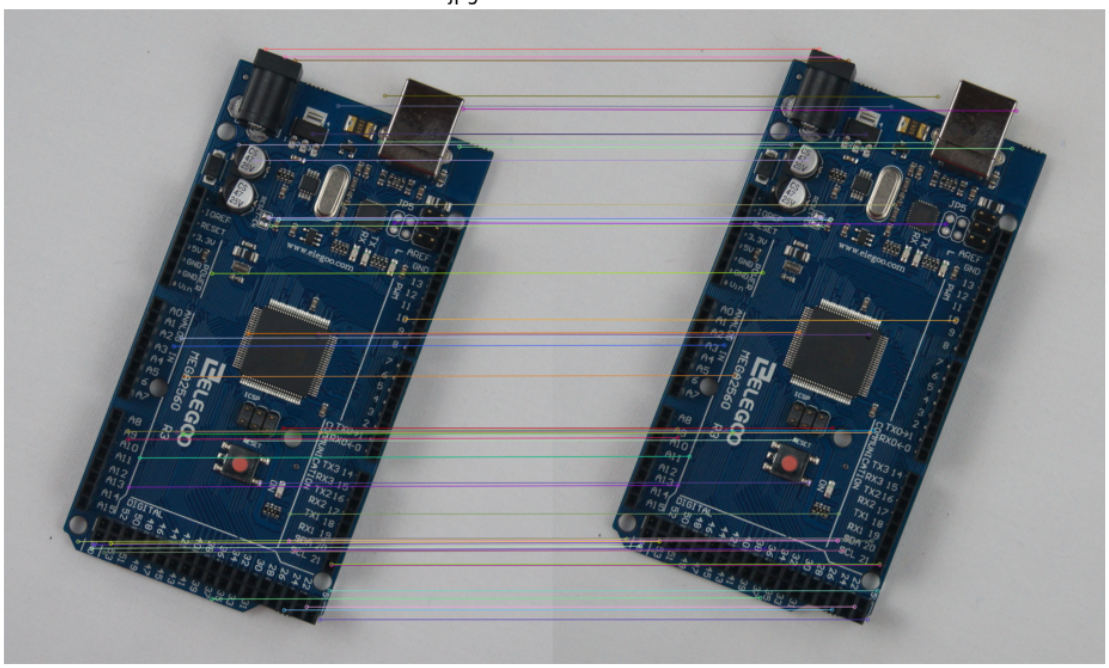
ADAD1.jpg to AECC1: inlier matches



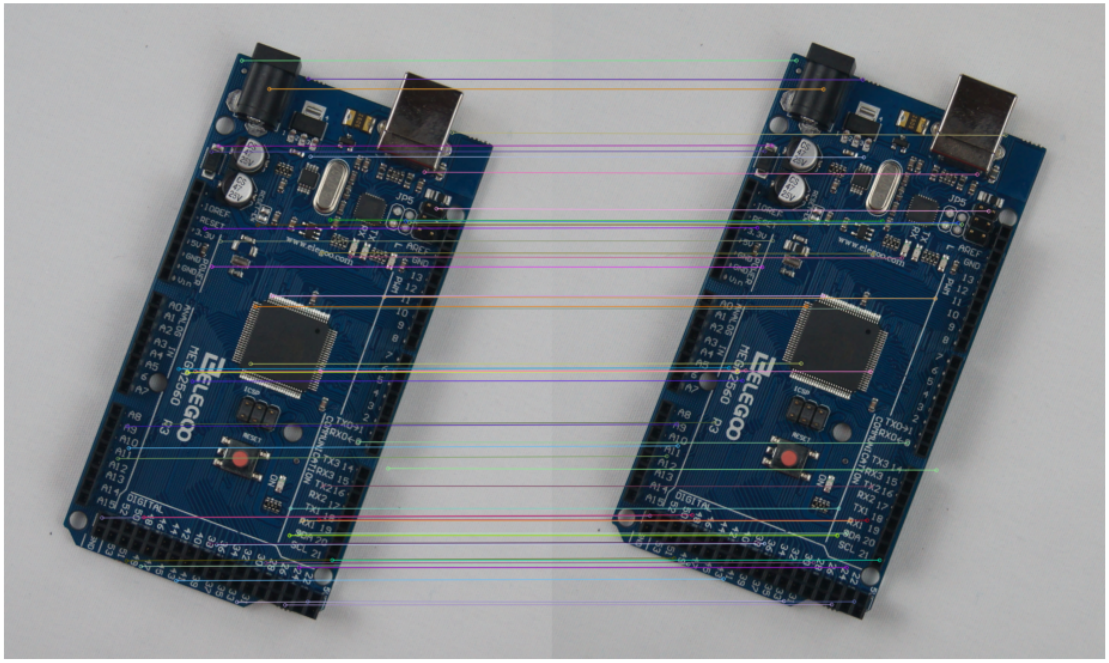
AECC2.jpg to AECC1: inlier matches



AECC3.jpg to AECC1: inlier matches



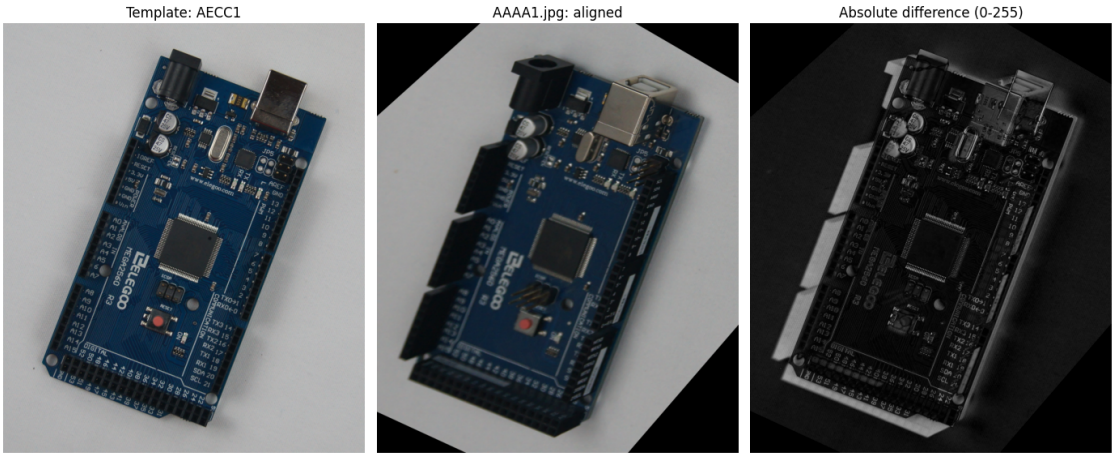
AECC4.jpg to AECC1: inlier matches



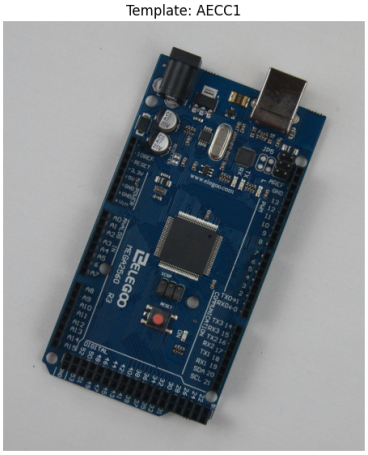
1.5.3 4(c) and 4(d) Warp all PCB images and display differences

Each image is warped into the template coordinates. The absolute colour difference is averaged across channels for display, using the same 0-255 range for every image. Invalid warp borders are masked out and excluded from the mean absolute error.

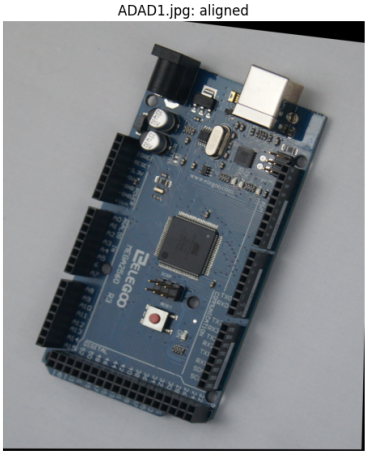
[48]:
```
size = (template.shape[1], template.shape[0])
for board in boards:
    aligned = cv.warpPerspective(board['image'], board['H'], size)
    valid = cv.warpPerspective(np.full(board['image'].shape[:2], 255, np.uint8),
                               board['H'], size, flags=cv.INTER_NEAREST)
    valid = cv.erode(valid, np.ones((5, 5), np.uint8)) > 0
    difference = cv.absdiff(aligned, template)
    difference[~valid] = 0
    board['mae'] = difference[valid].mean()
    board['coverage'] = valid.mean()
    fig, axes = plt.subplots(1, 3, figsize=(14, 6))
    show(template, 'Template: AECC1', axes[0])
    show(aligned, board['name'] + ': aligned', axes[1])
    show(difference.mean(axis=2).astype(np.uint8), 'Absolute difference␣
 ↪(0-255)', axes[2])
    plt.tight_layout()
    plt.show()
print('Image       Matches  Inliers  Median error(px)  MAE(0-255)  Coverage')
for b in boards:
    print(f"{b['name']:10s} {b['good']:7d} {b['inliers']:8d} {b['error']:16.2f}␣
 ↪{b['mae']:12.2f} {100*b['coverage']:8.1f}%")
print('AECC1 compared to itself has zero difference; it is the reference.')
```



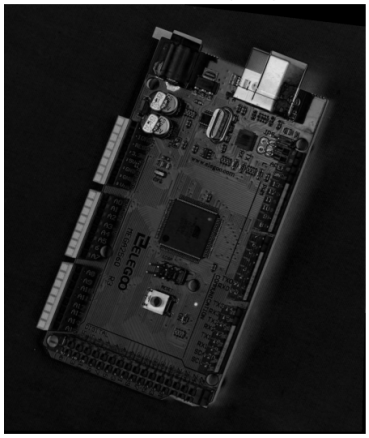
Template: AECC1



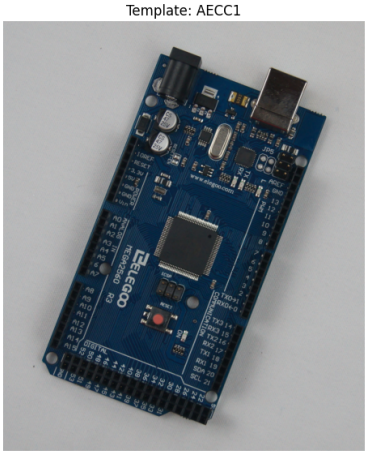
ADAD1.jpg: aligned



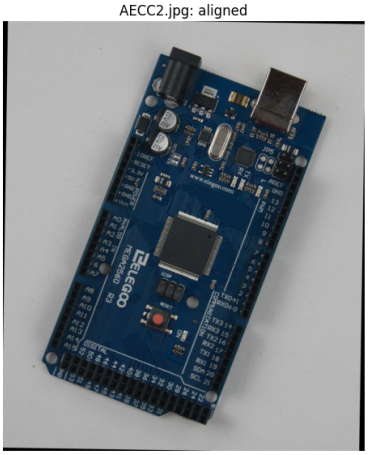
Absolute difference (0-255)



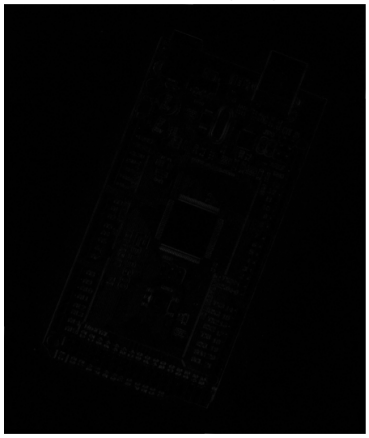
Template: AECC1



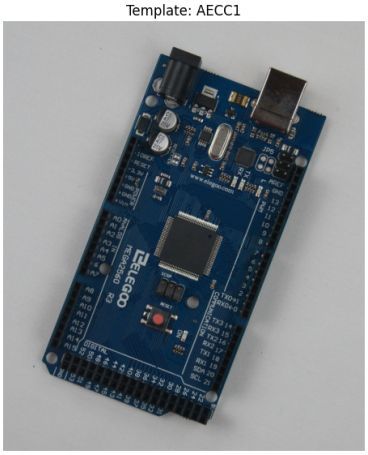
AECC2.jpg: aligned



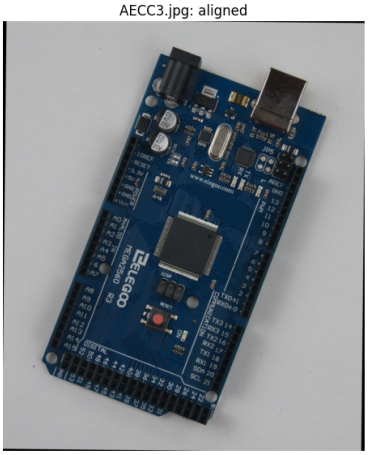
Absolute difference (0-255)



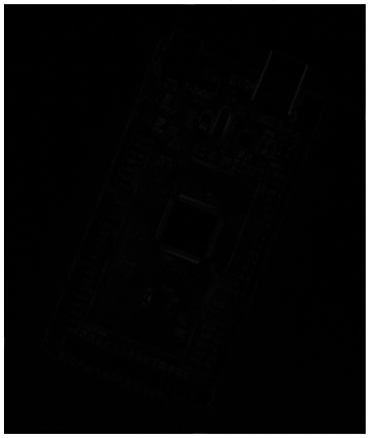
Template: AECC1



AECC3.jpg: aligned



Absolute difference (0-255)



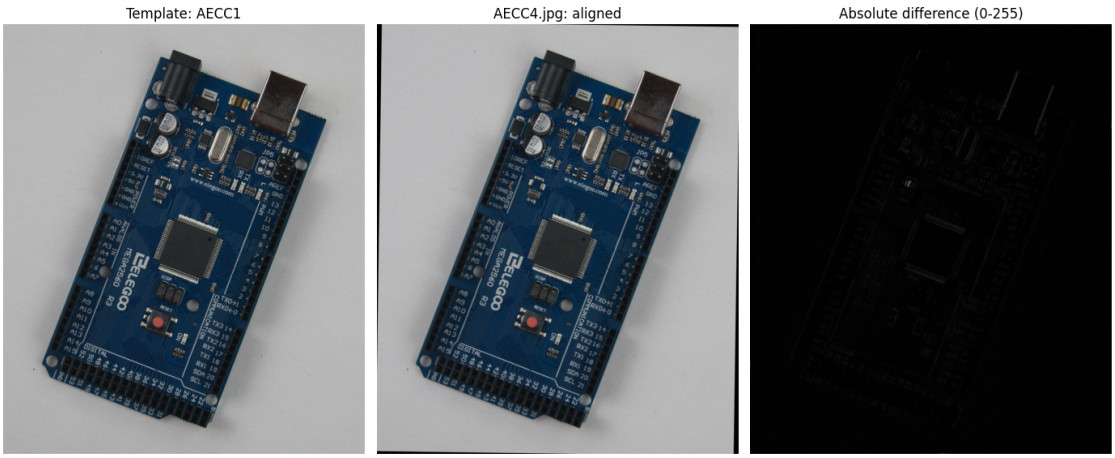


Image	Matches	Inliers	Median error(px)	MAE(0-255)	Coverage
AAAA1.jpg	54	26	0.96	28.85	88.8%
ADAD1.jpg	288	179	0.34	30.39	97.8%
AECC2.jpg	2722	2505	0.18	2.85	98.0%
AECC3.jpg	3131	2876	0.14	2.61	98.5%
AECC4.jpg	2783	2535	0.18	3.46	98.1%

AECC1 compared to itself has zero difference; it is the reference.

1.5.4 4(e) Discussion

The USB Type B port, capacitors and reset button extend above the PCB plane. Their apparent positions shift with the viewpoint differently from the flat board (parallax), so a single homography cannot align every component. Shadows, reflections, exposure and occlusion also produce differences. A bright difference region does not necessarily mean that a component is defective.

The median inlier reprojection error checks how well the matched features align. MAE also includes illumination and component differences, so it is not a defect count. Compare the more oblique AAAA1 and ADAD1 views with the similar AECC views using the table and difference images above.